



254383 Algorithm Design and Analysis

อ.พิเศษพงษ์ สุธาพันธ์
ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ





Week 2

หลักพื้นฐานของการวิเคราะห์ ประสิทธิภาพอัลกอริทึม

อ.พิเศษพงศ์ สุธาพันธ์

ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ



Analysis of algorithms

- **วัตถุประสงค์**
 - รู้ถึงวิธีการวัดประสิทธิภาพของอัลกอริทึม
 - สามารถระบุประสิทธิภาพทางด้านเวลาได้
 - สามารถเปรียบเทียบประสิทธิภาพระหว่างอัลกอริทึมได้

Analysis of algorithms

- ประเด็น (Issues)
 - ความถูกต้อง (correctness)
 - ประสิทธิภาพด้านเวลา (time efficiency)
 - ประสิทธิภาพด้านพื้นที่ (space efficiency)
 - เหมาะสม (optimality)



Analysis of algorithms

- **วิธี (Approaches)**

- การวิเคราะห์เชิงทฤษฎี (theoretical analysis)

- การคำนวณ

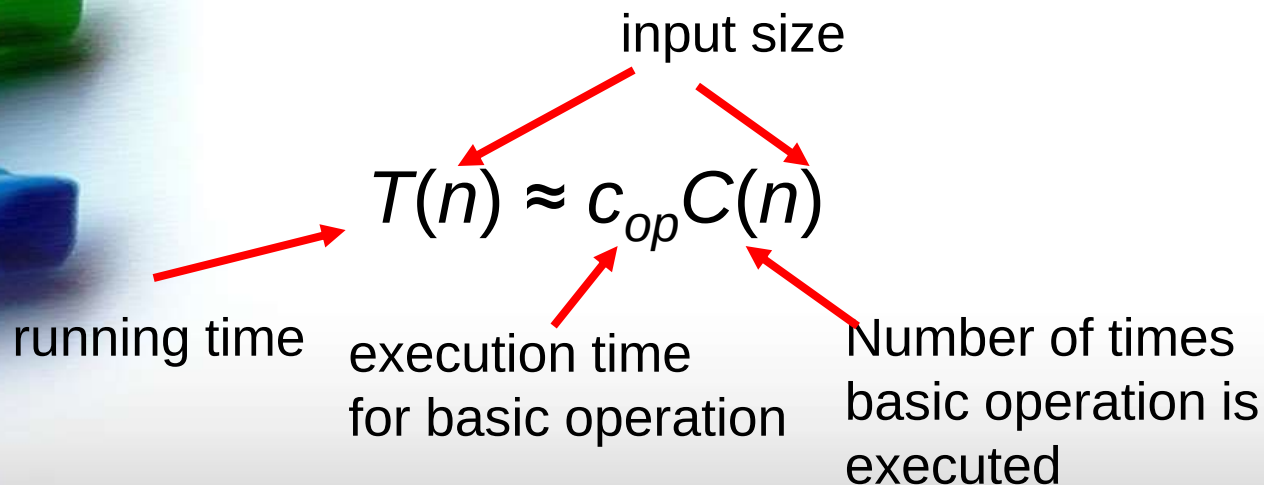
- การวิเคราะห์เชิงประจักษ์ (empirical analysis)

- การทดลอง

Theoretical analysis of time efficiency

Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size

- Basic operation: the operation that contributes most towards the running time of the algorithm



Input size and basic operation examples

<i>Problem</i>	<i>Input size measure</i>	<i>Basic operation</i>
Searching for key in a list of n items	Number of list's items, i.e. n	Key comparison
Multiplication of two matrices	Matrix dimensions or total number of elements	Multiplication of two numbers
Checking primality of a given integer n	n'size = number of digits (in binary representation)	Division
Typical graph problem	#vertices and/or edges	Visiting a vertex or traversing an edge

Empirical analysis of time efficiency

- Select a specific (typical) sample of inputs
- Use physical unit of time (e.g., milliseconds)
or
Count actual number of basic operation's executions
- Analyze the empirical data

Best-case, average-case, worst-case

For some algorithms efficiency depends on form of input:

- Worst case: $C_{\text{worst}}(n)$ – maximum over inputs of size n
- Best case: $C_{\text{best}}(n)$ – minimum over inputs of size n
- Average case: $C_{\text{avg}}(n)$ – “average” over inputs of size n
 - Number of times the basic operation will be executed on typical input
 - NOT the average of worst and best case
 - Expected number of basic operations considered as a random variable under some assumption about the probability distribution of all possible inputs



การวิเคราะห์(Analysis)

Analysis

การแบ่งสาระสำคัญออกเป็นส่วนๆเพื่อ
ทำการศึกษาเจาะจงเป็นส่วนๆ



การวิเคราะห์(Analysis)

Analysis of algorithm

เน้น การศึกษาประสิทธิภาพของ
อัลกอริทึม

- Running time
- Memory space



การวิเคราะห์(Analysis)

สัญลัษณ์(Notation) ที่สำคัญ

- O (“big oh”)
- Ω (“big omega”)
- Θ (“big theta”)



The Analysis Framework

- Time efficiency
- Space efficiency



The Analysis Framework

Time efficiency

หรือที่เรียกว่า Time complexity

บอกถึงความเร็วของอัลกอริทึมในการหาคำตอบ



The Analysis Framework

Space efficiency

หรือที่เรียกว่า **Space complexity**

อ้างอิงจำนวนหน่วยความจำทั้งหมดที่ถูกใช้โดย
อัลกอริทึมที่ต้องการพื้นที่เพิ่มทั้งในส่วนของ
input และ output



The Analysis Framework

Measuring an Input's Size

จากการสังเกต อัลกอริทึมส่วนมากที่ใช้เวลา
ทำงานนานกว่าเนื่องจากมี input ที่มากกว่า



The Analysis Framework

Units for Measuring Running Time

นับเวลาที่ใช้ในการทำงานของตัวดำเนินการ
(basic operation) มาเป็นตัววัด

ตย. ในการจัดเรียงหรือการค้นหา ตัวดำเนินการ
คือ การเปรียบเทียบ

ตย. ในการคำนวณ ตัวดำเนินการ คือ การบวก
การลบ การคูณ และ การหาร



The Analysis Framework

การประมาณเวลาที่ใช้ในการทำงาน

ให้ c_{op} คือเวลาที่ใช้ในการทำงานของ basic operation

ให้ $C(n)$ คือจำนวนครั้งที่เรียกใช้ basic operation ใน อัลกอริทึม

ให้ $T(n)$ คือเวลาที่ใช้โดยประมาณของอัลกอริทึม

$$T(n) \approx c_{op} C(n)$$



The Analysis Framework

ตย. สมมติ $C(n) = \frac{1}{2}n(n - 1) \approx \frac{1}{2}n^2$

ถ้า จำนวนการทำงานเพิ่มขึ้น 2 เท่า คือ $2n$ แล้วเวลาจะเพิ่มขึ้นเท่าไร

$$\frac{T(2n)}{T(n)} = \frac{c_{op}C(2n)}{c_{op}C(n)} = \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = 4$$



The Analysis Framework

Orders of Growth

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 * 10^1$	10^2	10^3	10^3	$3.6 * 10^6$
10^2	6.6	10^2	$6.6 * 10^2$	10^4	10^6	$1.3 * 10^{30}$	$9.3 * 10^{157}$
10^3	10	10^3	$1.0 * 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 * 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 * 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 * 10^7$	10^{12}	10^{18}		



The Analysis Framework

Efficiencies

- **Worst-Case**
- **Best-Case**
- **Average-Case**



The Analysis Framework

Algorithm SequentialSearch($A[0..n-1], K$)

$i = 0$

while $i < n$ and $A[i] \neq K$ do

$i = i + 1$

if $i < n$ return i

else return -1

- **Worst case**
- **Best case**
- **Average case**



The Analysis Framework

จากอัลกอริทึม

กรณี worst-case

ไม่มีค่า K ในอาร์เรย์ A



The Analysis Framework

จากอัลกอริทึม

กรณี best-case

ค่า K ในอาร์เรย์ A ตำแหน่งแรก



The Analysis Framework

จากอัลกอริทึม

กรณี average-case

$$\begin{aligned} C_{avg}(n) &= \left[1 \frac{p}{n} + 2 \frac{p}{n} + \cdots + n \frac{p}{n} \right] + n(1 - p) \\ &= \frac{p}{n} [1 + 2 + \cdots + n] + n(1 - p) \\ &= \frac{p}{n} \frac{n(n+1)}{2} + n(1 - p) \\ &= \frac{p(n+1)}{2} + n(1 - p) \end{aligned}$$

ให้ $p = 1$

$$\frac{1+2+3+\dots+n}{n} = \frac{n(n+1)}{2n} = \frac{(n+1)}{2}$$

Types of formulas for basic operation's count

- Exact formula

$$\text{e.g., } C(n) = n(n-1)/2$$

- Formula indicating order of growth with specific multiplicative constant

$$\text{e.g., } C(n) \approx 0.5 n^2$$

- Formula indicating order of growth with unknown multiplicative constant

$$\text{e.g., } C(n) \approx cn^2$$



The Analysis Framework

กรอบที่ใช้ในการวิเคราะห์

- ทั้งประสิทธิภาพทั้งเวลาและเนื้อที่ วัดได้จากขนาดข้อมูลที่น่าเข้า
- เวลาจะวัดโดยการนับจำนวนครั้งของ basic operation
- หาความแตกต่างระหว่าง worst-case , best-case และ average-case
- สนใจลำดับของการเจริญเติบโตเมื่อข้อมูลมีจำนวนมากจนถึงinfinity



Asymptotic Notations and Basic Efficiency Classes

สัญลักษณ์(Notation) ที่สำคัญ

- O (“big oh”)
 - Ω (“big omega”)
 - Θ (“big theta”)
- ❖ $t(n)$ เป็น เวลาที่ใช้อัลกอริทึมทำงาน โดยนับจาก basic operation $C(n)$
- ❖ $g(n)$ เป็น ฟังก์ชันพื้นฐานที่เป็นตัวเปรียบเทียบเวลาการทำงาน



Asymptotic Notations and Basic Efficiency Classes

$O(g(n))$ คือทุกฟังก์ชันที่มีอยู่ในลำดับการเติบโตนี้ จะมีลำดับการเติบโตน้อยกว่าหรือเท่ากับของ $g(n)$

$$n \in O(n^2)$$

$$100n + 5 \in O(n^2)$$

$$\frac{1}{2}n(n - 1) \in O(n^2)$$



Asymptotic Notations and Basic Efficiency Classes

$O(g(n))$ คือทุกฟังก์ชันที่มีอยู่ในลำดับการเติบโตนี้ จะมีลำดับการเติบโตน้อยกว่าหรือเท่ากับของ $g(n)$

$$n^3 \notin O(n^2)$$

$$0.0001n^3 \notin O(n^2)$$

$$n^4 + n + 1 \notin O(n^2)$$



Asymptotic Notations and Basic Efficiency Classes

$\Omega(g(n))$ คือทุกฟังก์ชันที่อยู่ในลำดับการเติบโตนี้ จะมีลำดับการเติบโตมากกว่าหรือเท่ากับของ $g(n)$

$$n^3 \in \Omega(n^2)$$

$$\frac{1}{2}n(n-1) \in \Omega(n^2)$$

$$100n + 5 \notin \Omega(n^2)$$



Asymptotic Notations and Basic Efficiency Classes

$\Theta(g(n))$ คือทุกฟังก์ชันที่อยู่ในลำดับการเติบโตนี้ จะมีลำดับการเติบโตเท่ากับของ $g(n)$

$$an^2 + bn + c \in \Theta(n^2)$$



Asymptotic Notations and Basic Efficiency Classes

O-notation

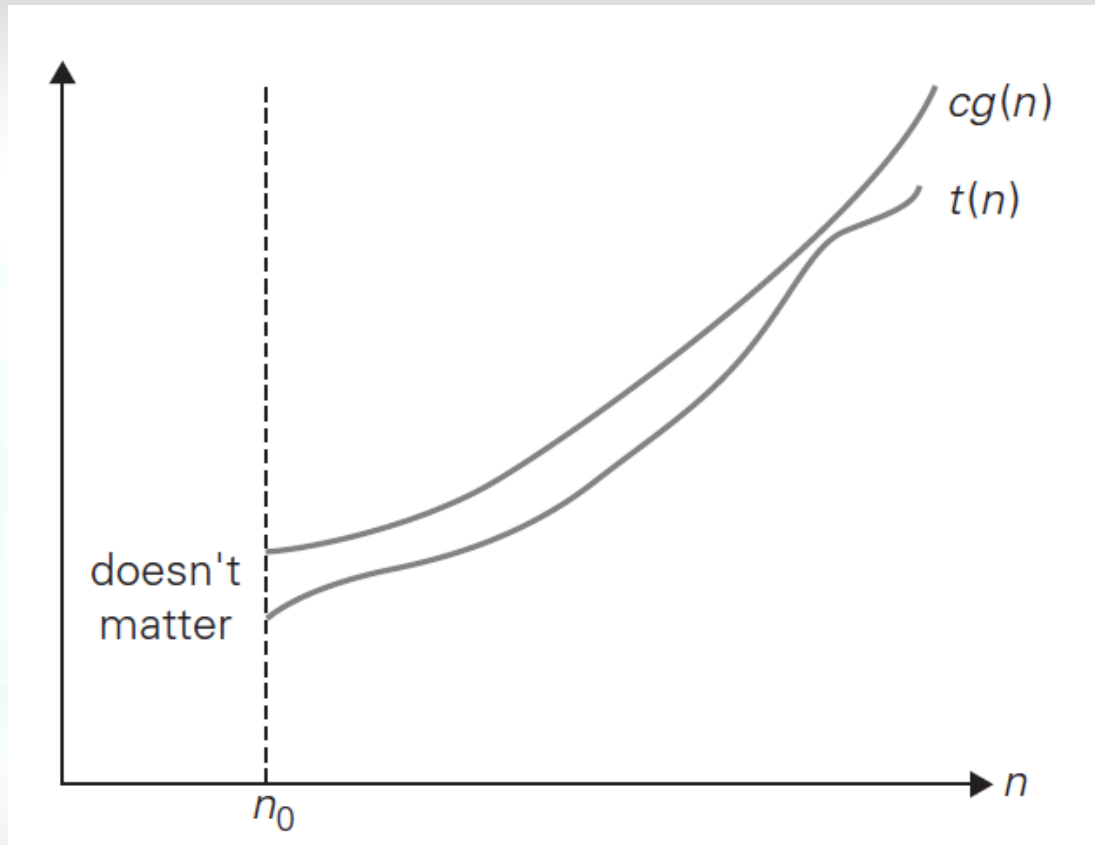
$t(n)$ อยู่ใน $O(g(n))$, แสดงว่า $t(n) \in O(g(n))$, ถ้า $t(n)$ มีขอบเขตที่น้อยกว่า $g(n)$ คูณด้วยค่าคงที่ สำหรับทุกค่าที่มากกว่า n

$$t(n) \leq cg(n); n \geq n_0$$



Asymptotic Notations and Basic Efficiency Classes

O-notation



$$t(n) \leq cg(n); n \geq n_0$$



Asymptotic Notations and Basic Efficiency Classes

O-notation

ต.ย. $100n + 5 \in O(n^2)$

$$100n + 5 \leq 100n + n ; n \geq 5$$

$$100n + 5 \leq 101n ; n \geq 5$$

$$100n + 5 \leq 101n^2 ; n \geq 5$$



Asymptotic Notations and Basic Efficiency Classes

O-notation

ต.ย. $100n + 5 \in O(n^2)$

เราต้องการหา c และ n_0

$$100n + 5 \leq 100n + 5n ; n \geq 1$$

$$100n + 5 \leq 105n$$

$$100n + 5 = 105n ; n = 1$$

$$\therefore c = 105 ; n_0 = 1$$



Asymptotic Notations and Basic Efficiency Classes

Ω -notation

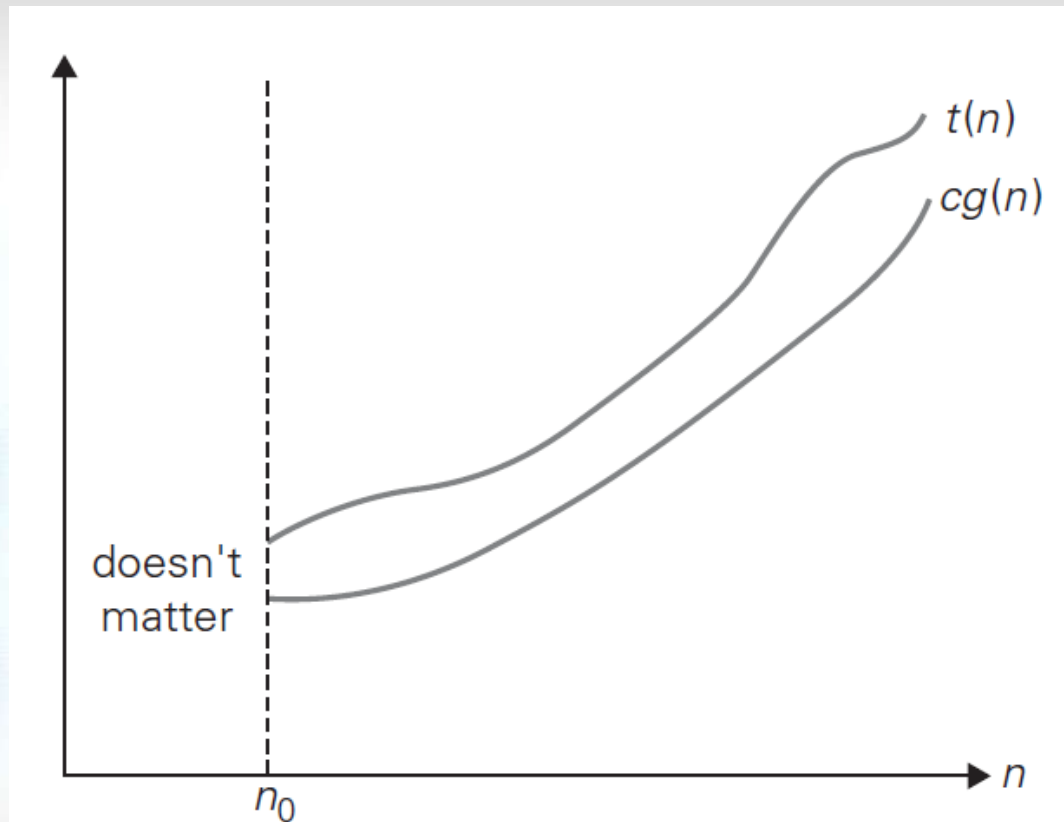
$t(n)$ อยู่ใน $\Omega(g(n))$, แสดงว่า $t(n) \in \Omega(g(n))$, ถ้า $t(n)$ มีขอบเขตที่มากกว่า $g(n)$ คูณด้วยค่าคงที่ สำหรับทุกค่าที่มากกว่า n

$$t(n) \geq cg(n); n \geq n_0$$



Asymptotic Notations and Basic Efficiency Classes

Ω -notation



$$t(n) \geq cg(n); n \geq n_0$$



Asymptotic Notations and Basic Efficiency Classes

Ω -notation

$$\text{ত.য. } n^3 \in \Omega(n^2)$$

$$n^3 \geq n^2 ; n \geq 0$$

$$\therefore c = 1 ; n_0 = 0$$



Asymptotic Notations and Basic Efficiency Classes

Θ -notation

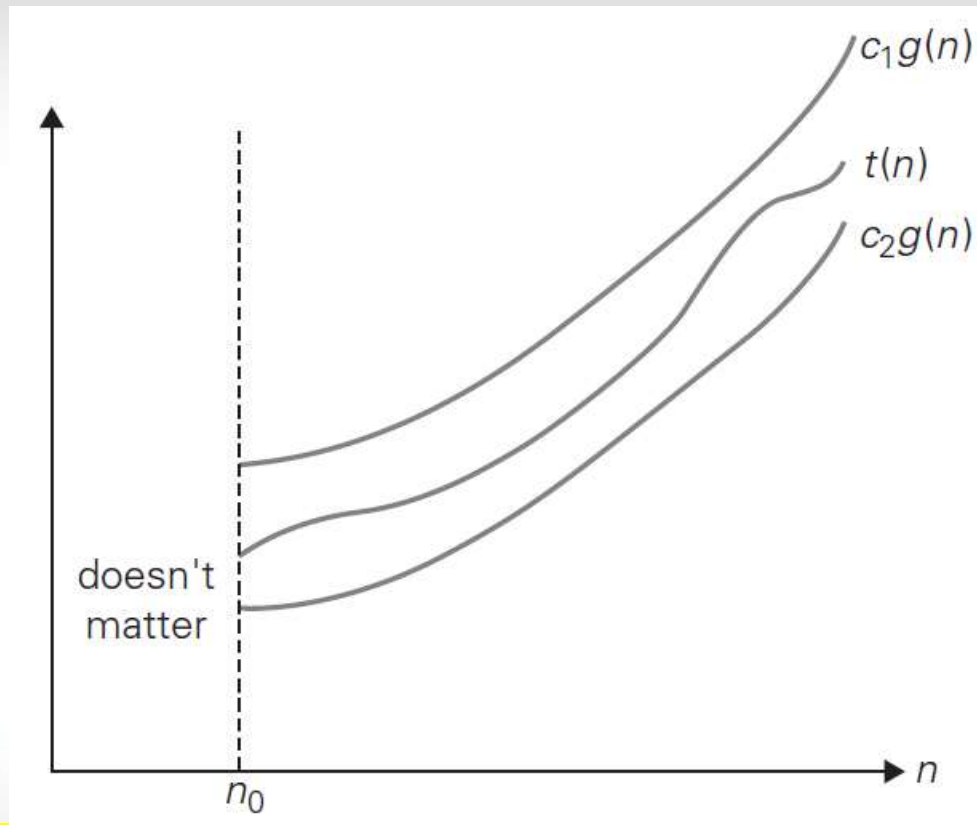
$t(n)$ อยู่ใน $\Theta(g(n))$, แสดงว่า $t(n) \in \Theta(g(n))$, ถ้า $t(n)$ มีขอบเขตที่ระหว่าง $g(n)$ คูณด้วยค่าคงที่ c_1 และ c_2 สำหรับทุกค่าที่มากกว่า n

$$c_2 g(n) \leq t(n) \leq c_1 g(n); n \geq n_0$$



Asymptotic Notations and Basic Efficiency Classes

Θ -notation



$$c_2g(n) \leq t(n) \leq c_1g(n); n \geq n_0$$



Asymptotic Notations and Basic Efficiency Classes

Θ -notation

ต.ย. $\frac{1}{2}n(n-1) \in \Theta(n^2)$

ทำด้านขวา

$$\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2 ; n \geq 0$$

ทำด้านซ้าย

$$\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{2}n^2 - \frac{1}{2}n \frac{1}{2}n = \frac{1}{4}n^2 ;$$

$n \geq 2$



Asymptotic Notations and Basic Efficiency Classes

Θ -notation

$$\text{ต.ย. } \frac{1}{2}n(n-1) \in \Theta(n^2)$$

$$\therefore c_1 = \frac{1}{2} ; c_2 = \frac{1}{4} ; n_0 = 2$$



Asymptotic Notations and Basic Efficiency Classes

คุณสมบัติ Asymptotic notations

THEOREM : if $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$

$$t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$$



Asymptotic Notations and Basic Efficiency Classes

ใช้ลิมิตในการเปรียบเทียบลำดับการโต

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \begin{cases} 0 \\ c \\ \infty \end{cases}$$

- 0 implies that $t(n)$ has a smaller order of growth than $g(n)$,
- c implies that $t(n)$ has the same order of growth as $g(n)$,
- ∞ implies that $t(n)$ has a larger order of growth than $g(n)$.³



Asymptotic Notations and Basic Efficiency Classes

ใช้ลิมิตในการเปรียบเทียบลำดับการโต

ต.ย. $t(n) = n^2$ และ $g(n) = n^3$

$$\lim_{n \rightarrow \infty} \frac{n^2}{n^3}$$

0 implies that $t(n)$ has a smaller order of growth than $g(n)$,



Asymptotic Notations and Basic Efficiency Classes

ใช้ลิมิตในการเปรียบเทียบลำดับการโต

ต.ย. $t(n) = 2n^2$ และ $g(n) = n^2$

$$\lim_{n \rightarrow \infty} \frac{2n^2}{n^2}$$

c implies that $t(n)$ has the same order of growth as $g(n)$,



Asymptotic Notations and Basic Efficiency Classes

ใช้ลิมิตในการเปรียบเทียบลำดับการโต

ต.ย. $t(n) = n^3$ และ $g(n) = n^2$

$$\lim_{n \rightarrow \infty} \frac{n^3}{n^2}$$

∞ implies that $t(n)$ has a larger order of growth than $g(n)$

การวิเคราะห์ทาง
คณิตศาสตร์สำหรับ
อัลกอริทึมที่ไม่มีการเรียกใช้
ตัวเอง



Mathematical Analysis of Nonrecursive Algorithms

๓.๒. 1

ALGORITHM *MaxElement*($A[0..n - 1]$)

//Determines the value of the largest element in a given array

//Input: An array $A[0..n - 1]$ of real numbers

//Output: The value of the largest element in A

$maxval \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $A[i] > maxval$

$maxval \leftarrow A[i]$

return $maxval$

$$C(n) = \sum_{i=1}^{n-1} 1.$$



Mathematical Analysis of Nonrecursive Algorithms

๓.๒. 1

$$C(n) = \sum_{i=1}^{n-1} 1.$$

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n).$$



Mathematical Analysis of Nonrecursive Algorithms

แนวทางวิเคราะห์ประสิทธิภาพอัลกอริทึมแบบ Nonrecursive

1. หาขนาดของ input
2. กำหนด basic operation
3. หาจำนวนการทำงานของ basic operation
4. เขียน sum expressing ของ basic operation
5. ใช้สูตรและกฎของ sum manipulation



Mathematical Analysis of Nonrecursive Algorithms

กฎพื้นฐานของ sum of manipulation

$$\sum_{i=l}^u ca_i = c \sum_{i=l}^u a_i$$

$$\sum_{i=l}^u (a_i \pm b_i) = \sum_{i=l}^u a_i \pm \sum_{i=l}^u b_i$$



Mathematical Analysis of Nonrecursive Algorithms

สูตรผลรวม

$$\sum_{i=l}^u 1 = u - l + 1 \quad l \leq u$$

$$\sum_{i=0}^n i = \sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2)$$



Mathematical Analysis of Nonrecursive Algorithms

๓.๒. 2

ALGORITHM *UniqueElements*($A[0..n - 1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n - 1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] = A[j]$ **return false**

return true

$$C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$



Mathematical Analysis of Nonrecursive Algorithms

ต.ย. 2

$$C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

$$= \sum_{i=0}^{n-2} (n - 1 - i)$$

$$\sum_{i=0}^{n-2} (n - 1 - i) = (n - 1) + (n - 2) + \cdots + 1 = \frac{(n - 1)n}{2}$$



Mathematical Analysis of Nonrecursive Algorithms

๓.๒. 3

$$\begin{array}{ccc} A & B & C \\ \text{row } i \left[\begin{array}{|c|c|c|c|c|} \hline \square & \square & \square & \square & \square \\ \hline \end{array} \right] * \left[\begin{array}{|c|} \hline \square \\ \square \\ \square \\ \square \\ \square \\ \square \\ \hline \end{array} \right] & = & \left[\begin{array}{|c|} \hline C[i,j] \\ \hline \end{array} \right] \\ & \text{col. } j & \end{array}$$



Mathematical Analysis of Nonrecursive Algorithms

๓.๒. 3

ALGORITHM *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)
//Multiplies two square matrices of order n by the definition-based algorithm
//Input: Two $n \times n$ matrices A and B
//Output: Matrix $C = AB$
for $i \leftarrow 0$ **to** $n-1$ **do**
 for $j \leftarrow 0$ **to** $n-1$ **do**
 $C[i, j] \leftarrow 0.0$
 for $k \leftarrow 0$ **to** $n-1$ **do**
 $C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$
return C

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$



Mathematical Analysis of Nonrecursive Algorithms

๓.๒. 4

ALGORITHM *Binary*(n)

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

$count \leftarrow 1$

while $n > 1$ **do**

$count \leftarrow count + 1$

$n \leftarrow \lfloor n/2 \rfloor$

return $count$

$n = 16$

$n = 8$

$n = 4$

$n = 2$

$n = 1$



Mathematical Analysis of Nonrecursive Algorithms

๓.๒. 4

ALGORITHM *Binary(n)*

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

$count \leftarrow 1$

while $n > 1$ **do**

$count \leftarrow count + 1$

$n \leftarrow \lfloor n/2 \rfloor$

return $count$

$$t(n) = \lfloor \log_2 n \rfloor + 1$$

การวิเคราะห์ทาง
คณิตศาสตร์สำหรับ
อัลกอริทึมที่มีการเรียกใช้
ตัวเอง



Mathematical Analysis of Recursive Algorithms

ต.ย. 1

$$n! = 1 \cdot \dots \cdot (n - 1) \cdot n = (n - 1)! \cdot n \quad \text{for } n \geq 1$$

ALGORITHM $F(n)$

//Computes $n!$ recursively

//Input: A nonnegative integer n

//Output: The value of $n!$

if $n = 0$ **return** 1

else return $F(n - 1) * n$

สมการ recurrence

หาคำตอบจาก

$$F(n) = F(n - 1) \cdot n \quad \text{for } n > 0$$



Mathematical Analysis of Recursive Algorithms

ต.ย. 1

$$n! = 1 \cdot \dots \cdot (n - 1) \cdot n = (n - 1)! \cdot n \quad \text{for } n \geq 1$$

ALGORITHM $F(n)$

//Computes $n!$ recursively

//Input: A nonnegative integer n

//Output: The value of $n!$

if $n = 0$ **return** 1

else return $F(n - 1) * n$

สมการ recurrence

จำนวนครั้ง

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0$$

to compute $F(n-1)$ to multiply $F(n-1)$ by n



Mathematical Analysis of Recursive Algorithms

๓.๒. 1

Method of backward substitutions

$$M(n) = M(n-1) + 1$$

$$M(n-1) = M(n-2) + 1$$

$$M(n-2) = M(n-3) + 1$$

....

$$M(0) = 0$$

$$M(n) = M(n-1) + 1$$

$$M(n) = M(n-2) + 1 + 1$$

$$M(n) = M(n-3) + 1 + 1 + 1$$

...

$$M(n) = 0 + \dots + 1 + 1 + 1$$



Mathematical Analysis of Recursive Algorithms

แนวทางวิเคราะห์ประสิทธิภาพอัลกอริทึมแบบ Recursive

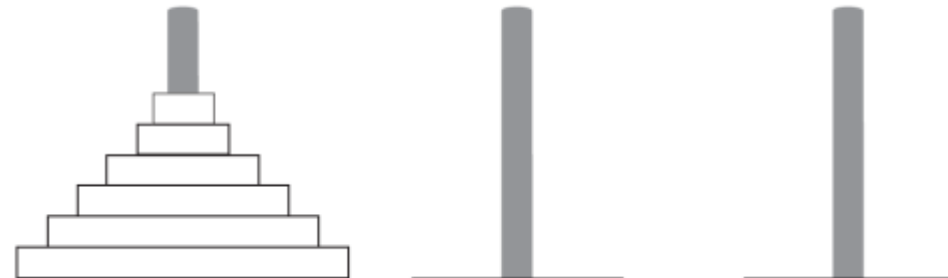
1. หาขนาดของ input
2. กำหนด basic operation
3. หาจำนวนการทำงานของ basic operation
4. กำหนดความสัมพันธ์แบบ recurrence
5. แก้สมการ recurrence



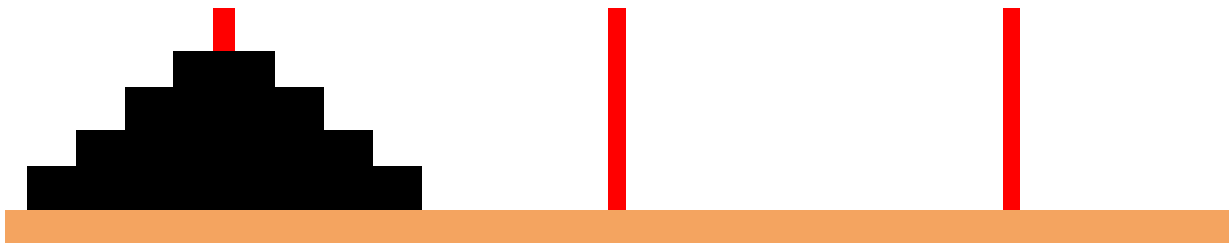
Mathematical Analysis of Recursive Algorithms

ต.ย. 2

Tower of Hanoi



<http://mathworld.wolfram.com/TowerofHanoi.html>



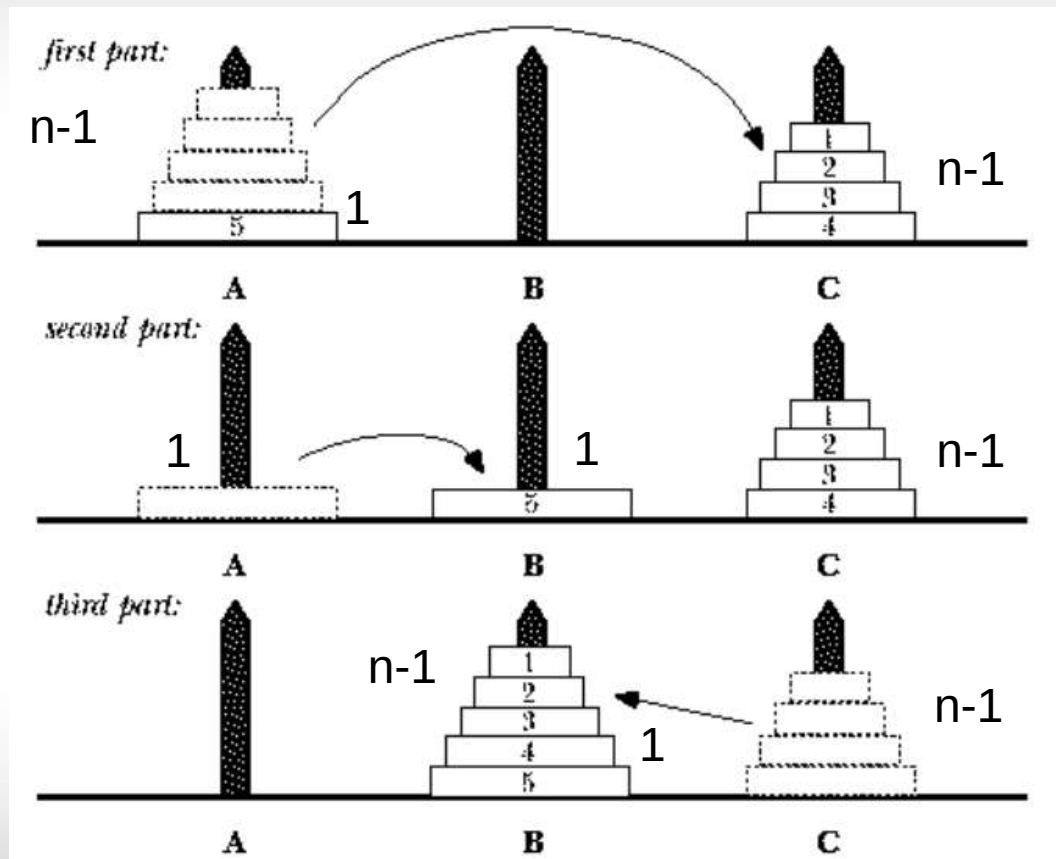
มีการเคลื่อนย้ายชิ้นไม้กี่ครั้ง?



Mathematical Analysis of Recursive Algorithms

ต.ย. 2

Tower of Hanoi





Mathematical Analysis of Recursive Algorithms

ต.ย. 2

Tower of Hanoi

สมการ recurrence

$$M(n) = M(n - 1) + 1 + M(n - 1) \quad \text{for } n > 1$$

$$M(n) = 2M(n - 1) + 1 \quad \text{for } n > 1$$

$$M(1) = 1.$$



Mathematical Analysis of Recursive Algorithms

๓.๒. 2

Tower of Hanoi

Method of backward substitutions

$$\begin{aligned}M(n) &= 2M(n-1) + 1 & \text{sub. } M(n-1) &= 2M(n-2) + 1 \\&= 2[2M(n-2) + 1] + 1 = 2^2M(n-2) + 2 + 1 & \text{sub. } M(n-2) &= 2M(n-3) + 1 \\&= 2^2[2M(n-3) + 1] + 2 + 1 = 2^3M(n-3) + 2^2 + 2 + 1. & &= 2^n - 1.\end{aligned}$$



Mathematical Analysis of Recursive Algorithms

ต.ย. 3

ALGORITHM *BinRec*(n)

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

if $n = 1$ **return** 1

else return *BinRec*($\lfloor n/2 \rfloor$) + 1

สมการ recurrence

$$A(1) = 0.$$

$$A(n) = A(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1.$$



Mathematical Analysis of Recursive Algorithms

ต.ย. 3

การลดครั้งละ $\frac{n}{2}$ ยากต่อการแก้สมการ

เปลี่ยนรูปเป็น

$$n = 2^k$$

ตามทฤษฎี Smoothness rule

สมการ recurrence ใหม่

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{for } k > 0$$

$$A(2^0) = 0$$



Mathematical Analysis of Recursive Algorithms

๓.๒. 3

Method of backward substitutions

$$A(2^k) = A(2^{k-1}) + 1$$

$$\text{substitute } A(2^{k-1}) = A(2^{k-2}) + 1$$

$$= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2$$

$$\text{substitute } A(2^{k-2}) = A(2^{k-3}) + 1$$

$$= [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3$$

...

...

$$= A(2^{k-i}) + i$$

...

$$= A(2^{k-k}) + k.$$

$$A(2^k) = A(1) + k = k$$



Mathematical Analysis of Recursive Algorithms

ต.ย. 3

$$n = 2^k \rightarrow k = \log_2 n$$

มาจาก

$$n = 2^k$$

$$\log n = \log 2^k$$

$$\log n = k \log 2$$

$$\frac{\log n}{\log 2} = k \rightarrow \log_2 n = k$$



Mathematical Analysis of Recursive Algorithms

๓.๒. 3

$$A(2^k) = A(1) + k = k,$$

$$n = 2^k \quad k = \log_2 n$$

$$A(n) = \log_2 n \in \Theta(\log n)$$



อ่านเพิ่มเติมด้วยตัวเอง

หัวข้อย่อย 2.5 – 2.7



ฝึกปฏิบัติ

เขียนโปรแกรมจาก pseudocoded ในบทเรียน



อ้างอิงและต้นฉบับ

[1] Introduction to the design & analysis of algorithm, 3rd,
Anany Levitin